

100% Offline 360° Photo Sphere Engine

Architecture, Sensor-Fusion Alignment, C++ Stitching Pipeline & Android/iOS Implementation

Core Objective: Replicate the complete functionality of *360 Photo Cam* entirely on-device without cloud servers, external network latency, or recurring costs. This guide covers orientation-guided target acquisition, real-time parallax minimization, low-memory feature tracking, spherical bundle adjustment, and equirectangular XMP generation.

1. Architectural Overview & Dataflow

Building a single-camera spherical capture tool offline requires managing mobile CPU and RAM limits. While cloud-backed solutions offload high-resolution processing to multi-core servers, on-device execution requires a multi-scale processing approach.

1. Real-Time Guided Capture

The IMU sensor fusion tracks device pose (q). An interactive 3D reticle displays target points on a virtual unit sphere. An auto-shutter triggers when alignment error $< 2.5^\circ$ and angular drift is near zero.

2. Image Downsampling & Pose Init

High-resolution shots (12MP) are captured to disk, but downsampled thumbnails (1.5MP–2MP) are fed into the matching pipeline to keep RAM consumption well below mobile OS limits.

3. Bundle Adjustment & Warping

Rotational homographies ($H = K R K^{-1}$) are optimized using ORB/AKAZE keypoints and Levenberg-Marquardt solvers, then mapped to an equirectangular coordinate system.

4. Seam Finding & Multi-Band Blending

Graph-cut seam estimation avoids ghosting caused by parallax or moving subjects, followed by two-band Laplacian blending to smooth brightness variations across frames.

2. Guided Capture: Virtual Sphere Lattice Design

To ensure complete $360^\circ \times 180^\circ$ coverage with adequate overlap (30–40% per shot), target coordinates are arranged on concentric pitch rings:

Elevation Ring (Pitch)	Yaw Increments	Target Count	Purpose
Equator (0°)	Every 36° (0°, 36°, 72°, ..., 324°)	10 Shots	Primary horizontal horizon
Upper Ring (+38°)	Every 45° (22.5°, 67.5°, ..., 337.5°)	8 Shots	Sky and upper structures
Lower Ring (-38°)	Every 45° (22.5°, 67.5°, ..., 337.5°)	8 Shots	Ground and floor coverage
Zenith (+90°)	Fixed Overhead Point	1 Shot	Full overhead closure
Nadir (-90°)	Fixed Ground Point	1 Shot	Ground closure (feet/tripod)

Total capture budget: 28 discrete frames. This balances spherical coverage against on-device stitching memory.

3. Sensor Fusion & Auto-Shutter Trigger Engine

The user interface requires an on-screen reticle guiding the user to rotate the phone about its optical center (no-parallax point). Hand translation must be minimized by displaying rotation hints.

3.1 Mathematical Formulation of Reticle Tracking

Let the target point in world coordinates be a unit vector $P_{world} = [x, y, z]^T$. Given the device rotation matrix R_{dev} derived from Android's `Sensor.TYPE_ROTATION_VECTOR` or iOS CoreMotion quaternion:

```
// 1. Transform World Target Vector to Camera Coordinate Space P_cam = R_dev * P_world; //
P_cam = [X_c, Y_c, Z_c]^T // 2. Reject points behind the camera lens if (Z_c <= 0) { /* Point is
behind phone; display directional guide arrow */ } // 3. Project to 2D Screen Space (Normalized
Device Coordinates -> Pixels) float u_screen = (X_c / Z_c) * (focal_length_x) + center_x; float
v_screen = (Y_c / Z_c) * (focal_length_y) + center_y;
```

Auto-Capture Trigger Conditions: Shutter executes automatically when:

1. **Angular Distance:** $\cos(\theta) = P_{cam} \cdot [0, 0, 1]^T \geq \cos(2.5^\circ)$ (dot is inside reticle).
2. **Stability Check:** Gyroscope magnitude $\|\omega\| < 0.08 \text{ rad/s}$ for 150ms (avoids motion blur).
3. **AE/AF Locked:** Exposure and white balance must be locked on shot #1 to prevent brightness banding.

4. OpenCV Native C++ Stitching Pipeline (NDK)

Below is the optimized C++ architecture utilizing OpenCV's stitching and detail pipelines, tuned specifically for mobile memory constraints.

```
#include <opencv2/opencv.hpp> #include <opencv2/stitching/detail/matchers.hpp> #include
<opencv2/stitching/detail/motion_estimators.hpp> #include
<opencv2/stitching/detail/exposure_compensators.hpp> #include
<opencv2/stitching/detail/seam_finders.hpp> #include <opencv2/stitching/detail/blenders.hpp>
cv::Mat StitchSphereOffline(const std::vector<cv::Mat>& full_images, const std::vector<cv::Vec4f>&
imu_quats) { int num_images = full_images.size(); // Step 1: Feature Detection on Downsampled
Frames (~1.2MP for speed/RAM) std::vector<cv::detail::ImageFeatures> features(num_images); auto
finder = cv::AKAZE::create(); // Robust & Patent-Free for (int i = 0; i < num_images; ++i) {
cv::Mat small; cv::resize(full_images[i], small, cv::Size(), 0.35, 0.35);
cv::detail::computeImageFeatures(finder, small, features[i]); features[i].img_idx = i; } // Step 2:
Pairwise Feature Matching with IMU-Constrained Neighborhood std::vector<cv::detail::MatchesInfo>
pairwise_matches; cv::detail::BestOf2NearestMatcher matcher(false, 0.65f, 6, 6); matcher(features,
pairwise_matches); // Step 3: Global Estimator & Bundle Adjustment
cv::detail::HomographyBasedEstimator estimator; std::vector<cv::detail::CameraParams> cameras;
estimator(features, pairwise_matches, cameras); for (size_t i = 0; i < cameras.size(); ++i) {
cv::Mat R; cameras[i].R.convertTo(R, CV_32F); cameras[i].R = R; } cv::detail::BundleAdjusterRay
adjuster; adjuster.setConfThresh(0.8); adjuster(features, pairwise_matches, cameras); ...
```

5. Spherical Warping, Seams & Blending

After solving camera rotations (R_i) and global focal length (f), frames are projected into an equirectangular canvas.

5.1 Equirectangular Unrolling & Seam Placement

The spherical coordinates (θ, ϕ) translate directly to canvas pixels (u, v) where $\theta \in [-\pi, \pi]$ and $\phi \in [-\pi/2, \pi/2]$.

```
// Step 4: Spherical Warper Initialization float warped_image_scale = static_cast<float>
(cameras[0].focal); auto warper = cv::makePtr<cv::SphericalWarper>(); // Step 5: Seam Optimization
via Graph Cut cv::detail::GraphCutSeamFinder
seam_finder(cv::detail::GraphCutSeamFinderBase::COST_COLOR); seam_finder.find(warped_images_small,
warped_masks_small, corners); // Step 6: Memory-Constrained Multi-Band Blender (2 Bands Max on
Mobile) auto blender = cv::detail::Blender::createDefault(cv::detail::Blender::MULTI_BAND, false);
cv::detail::MultiBandBlender* mb = dynamic_cast<cv::detail::MultiBandBlender*>(blender.get()); mb-
>setNumBands(2); // Prevents Out-Of-Memory (OOM) on devices with <= 4GB RAM mb-
>prepare(final_panorama_roi); for (int i = 0; i < num_images; ++i) { // Feed full-resolution frames
through calculated seam masks mb->feed(warped_full_img, warped_mask, corners[i]); } cv::Mat result,
result_mask; mb->blend(result, result_mask); return result; // Output: 2:1 Equirectangular Photo
Sphere }
```

6. Solving the Parallax, Nadir & Zenith Problems

The Handheld Parallax Challenge: When users rotate their hands rather than pivoting around the phone's physical camera lens, nearby objects shift against the background, causing double edges and broken lines.

Optical Pivot User Guidance

Display a live on-screen "pivot crosshair." Instruct users:
"Keep the phone in one point in space and rotate your body around the camera." This simple UX cue reduces translation error by over 70%.

Graph-Cut Edge Placement

The graph-cut seam solver dynamically steers cutlines around salient edges and moving obstacles, passing through low-gradient textures (walls, plain floors) where misalignment is unnoticeable.

Nadir (Ground) Inpainting

The ground shot (-90°) inevitably captures the user's feet and hands. Apply an elliptical alpha mask at the nadir pole combined with Telea's inpainting method (`cv::inpaint`) to fill the blind spot cleanly.

Zenith Sky Smoothing

Ceilings and clear skies often lack trackable features. Constrain the +90° zenith shot directly to the IMU gravity vector and use Poisson blending to close the top seam without requiring feature matches.

7. Google GPano XMP Metadata Injection

For standard photo galleries, Facebook, and VR viewers to automatically recognize the output as an interactive 360° photo sphere, the exported JPEG must contain the standard `http://ns.google.com/photos/1.0/panorama/` XMP block in its APP1 segment:

```
<x:xmpmeta xmlns:x="adobe:ns:meta/"> <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"> <rdf:Description rdf:about="" xmlns:GPano="http://ns.google.com/photos/1.0/panorama/">
<GPano:UsePanoramaViewer>True</GPano:UsePanoramaViewer>
<GPano:ProjectionType>equirectangular</GPano:ProjectionType>
<GPano:CroppedAreaImageWidthPixels>8192</GPano:CroppedAreaImageWidthPixels>
<GPano:CroppedAreaImageHeightPixels>4096</GPano:CroppedAreaImageHeightPixels>
<GPano:FullPanoWidthPixels>8192</GPano:FullPanoWidthPixels>
<GPano:FullPanoHeightPixels>4096</GPano:FullPanoHeightPixels>
<GPano:CroppedAreaLeftPixels>0</GPano:CroppedAreaLeftPixels>
<GPano:CroppedAreaTopPixels>0</GPano:CroppedAreaTopPixels> </rdf:Description> </rdf:RDF>
</x:xmpmeta>
```

8. Interactive 360° Viewer (OpenGL ES)

Displaying the finished spherical photo within your app without external dependencies requires a basic OpenGL ES 2.0/3.0 viewport:

- Inverted UV Sphere Mesh:** Generate an icosphere or UV sphere (48 rings, 48 sectors) centered at $(0, 0, 0)$. Invert triangle winding order so front faces point inward.
- Texture Mapping:** Bind the stitched equirectangular JPEG as a `GL_TEXTURE_2D` using linear sampling.
- Camera Controls:** Bind touch drag events $(\Delta X, \Delta Y)$ to camera yaw and pitch angles.
- Motion Sensor Look-Around:** Optional VR mode multiplying camera view matrix by the live device rotation matrix: $V' = R_{dev} \cdot T(0, 0, 0)$.

9. End-to-End Implementation Roadmap

Milestone	Primary Subsystem	Key Deliverables
Phase 1	Capture UI & Sensor Fusion	CameraX/AVFoundation preview, rotation vector tracking, auto-reticle dots.
Phase 2	Native OpenCV Integration	Build OpenCV C++ NDK library with AKAZE, Ray Adjuster, and Spherical Warper.
Phase 3	Memory Optimization	Two-stage pipeline: small thumbnails for BA/seams, single-pass blending for full res.
Phase 4	XMP Tagging & GL Viewer	APP1 JPEG metadata injector + OpenGL ES inverted sphere viewer component.